

Boundary conditions in a discontinuous-Galerkin solver

How the weak form becomes matrix blocks, and what the flags select

1 What a boundary condition *is* here

Take a conservation law

$$\partial_t f = -\partial_x F, \quad F = F(f, \partial_x f),$$

and a mesh of cells $I_j = [x_{j-1/2}, x_{j+1/2}]$. On each cell, multiply by a test function v and integrate by parts once:

$$\int_{I_j} v \partial_t f = \int_{I_j} v' F - \left[v \widehat{F} \right]_{x_{j-1/2}}^{x_{j+1/2}}. \quad (1)$$

The solution is discontinuous across cell edges, so F has two values there and the scheme must pick one. That single number, $\widehat{F}$, is the *numerical flux*. It is the only way neighbouring cells communicate.

Everything about boundary conditions follows from one observation about (1). Put $v \equiv 1$ (it is in the space) and sum over all cells. The volume term dies because $v' = 0$. Every *interior* edge appears twice — once as the right edge of I_j , once as the left edge of I_{j+1} — with opposite signs and, crucially, the *same* $\widehat{F}$, because the numerical flux is single-valued. They cancel in pairs. What survives is

$$\frac{d}{dt} \int f = \widehat{F}(a) - \widehat{F}(b) \quad (2)$$

for a domain $[a, b]$.

This is exact algebra, not an approximation: it holds at any mesh size and any polynomial degree. The interior of the scheme cannot create or destroy anything. *All* gain and loss enters through two numbers, the numerical fluxes at the two ends. So in a DG code:

A boundary condition is a rule for choosing $\widehat{F}$ at the two domain edges, where there is no neighbour to supply one.

Note what this is *not*. It is not a condition on f , and not a condition on $\partial_x f$. It is a condition on a flux. Those coincide only in special cases, and confusing them is the most common way to get a wrong wall.

2 Where the matrix blocks come from

Assembling (1) over a basis of local polynomials gives, per cell, a small dense block from the volume integral, plus blocks from the two edge terms. An edge term couples the cell to the trace of the solution on that edge; since each edge has a value from either side, it produces one block on the diagonal (this cell's own trace) and one off-diagonal (the neighbour's). The result is block-tridiagonal. The edge blocks carry a factor $1/h$: a trace evaluation of a basis function normalised on a cell of width h .

Now specialise to the first and last cell. Their outward edges have no neighbour, so the off-diagonal partner does not exist and only one block is in question. There are two things the assembler can do with it, and they are the two options a flag selects:

option	block	meaning
<i>free</i>	written, using this cell's own trace	$\widehat{F} = F(\text{interior})$
<i>fixed</i>	omitted	$\widehat{F} = 0$

The second line is worth pausing on. Prescribing zero flux is implemented by *not writing anything*: a zero contributes nothing to the matrix. The boundary condition is an absence. (A nonzero prescribed flux is a known number rather than an unknown, so it goes to the right-hand side as a source instead.)

Substituting into (2): a fixed edge contributes nothing to $d\int f/dt$, and a free edge contributes whatever flux the solution happens to carry there — an open door, whose flow is dynamic and unconstrained.

3 First-order terms: one flux, one choice

For an advective term $F = c f$ this is the whole story. Fix the edge and the wall is sealed; leave it free and material crosses at rate $c f(\text{wall})$.

The sign matters more than it first appears. If c points *out* of the domain at that wall, a free edge is an outflow condition: material leaves, $\int f$ decreases. If c points *in*, a free edge does not drain the domain — it *fills* it, because the scheme evaluates the flux as though the solution continued smoothly past the wall, and that fictitious outside region pushes its own material in. An open inflow boundary is a source. It is not a small error and refining the mesh does not reduce it.

4 Second-order terms: two traces, and why the flags come in pairs

A diffusive term $F = D \partial_x f$ cannot be handled the same way, because the weak form in (1) has already used its one integration by parts and $\partial_x f$ is still sitting there, undefined across cell edges. The standard fix (local discontinuous Galerkin) is to split the second-order operator into two first-order ones by naming the flux as an auxiliary unknown:

$$q = D \partial_x f, \quad \partial_t f = -\partial_x q. \quad (3)$$

Each of these is now first-order, and *each gets its own weak form with its own integration by parts*:

$$\int_{I_j} w q = -\int_{I_j} w' D f + [w D \widehat{f}], \quad \int_{I_j} v \partial_t f = \int_{I_j} v' q - [v \widehat{q}].$$

Two integrations by parts produce **two independent numerical traces**, and they are traces of different things: $\widehat{f}$, the value, appears only in the equation defining q ; and $\widehat{q}$, the flux, appears only in the equation advancing f .

This is the structural fact that governs everything else. In the assembled operator the two stages appear as a matrix product,

$$A = A_{\text{div}} A_{\text{grad}},$$

with q eliminated algebraically — it never exists as stored data, so it cannot be given a boundary condition of its own. Each factor owns exactly one of the two traces:

factor	owns the trace	fixing it imposes
the <i>outer</i> one (div)	$\hat{q}$	zero <i>flux</i> at that wall
the <i>inner</i> one (grad)	$\hat{f}$	zero <i>value</i> at that wall

A second-order operator admits exactly one boundary condition per wall. Hence the rule, and its two failure modes:

div	grad	result at that wall
fixed	free	Neumann: zero flux
free	fixed	Dirichlet: $f = 0$
fixed	fixed	over-determined — two conditions, ill-posed
free	free	under-determined — no condition

The two middle rows are real, usable boundary conditions. The two outer ones are not boundary conditions at all: they are malformed operators, and a solver will happily assemble either and produce nonsense — the over-determined case by diverging, the under-determined one by drifting.

The rule is per wall, not per term: you may fix the **div** at one end and the **grad** at the other, giving a domain that reflects on the left and absorbs on the right.

A caution about vocabulary. Some libraries flip the flag internally on the inner factor so that the same keyword means “impose my condition here” on both. That is convenient but means the two factors carry *opposite* literal settings for a well-posed pair. Check the convention of the code you are using before trusting a flag name.

5 Several terms: only the unsealed sum leaks

Real equations are sums of terms, $F = F_1 + F_2 + \dots$, each assembled separately and each carrying its own boundary choice. Because they share test functions, the ledger (2) sees only the total:

$$\frac{d}{dt} \int f = \sum_{k \text{ free}} [\hat{F}_k(a) - \hat{F}_k(b)]. \quad (4)$$

Sealed terms drop out. Two consequences, both practical:

Sealing a subset imposes one condition on their sum. If you seal terms 1 and 2 and leave 3 free, the wall enforces $F_1 + F_2 \rightarrow 0$ — not $F_1 \rightarrow 0$ and $F_2 \rightarrow 0$ separately — while F_3

leaks. So sealing a drag term Af together with a diffusion term $D\partial_x f$ does not give $\partial_x f = 0$; it gives

$$Af + D\partial_x f = 0 \quad \implies \quad \frac{\partial_x f}{f} = -\frac{A}{D},$$

a Robin condition that nobody wrote as one. For a Fokker–Planck operator this is exactly the equilibrium slope, which is why sealing everything reproduces the Maxwellian at the wall without anyone imposing it.

The audit. To find out what wall your solver actually has: list the terms; write each one’s flux; note which are sealed. The sealed ones give your boundary condition, as a sum. The unsealed ones, summed, are your leak rate — evaluated at the wall value of f , with sign. If that sum is not what you intended, the wall is quietly doing something else, usually without any warning from the code.

A flag acts on one term only

This is worth making explicit, because it is the reason the previous paragraph works. The discrete operator is assembled term by term. Each term contributes its own bilinear form, built from its own integration by parts, and they are added:

$$a(f, v) = \sum_k a_k(f, v), \quad a_k(f, v) = \int_{\Omega} v' F_k - \sum_{\text{edges}} [v \widehat{F}_k]. \quad (5)$$

The boundary choice for term k selects $\widehat{F}_k$ at the two domain edges. It therefore appears in exactly one summand of (5) — changing the flag on term k changes a_k and leaves every $a_{j \neq k}$ untouched, block for block. Nothing in the assembler ever compares one term’s flag with another’s, or checks that the collection is sensible.

The terms meet only afterwards, when $v \equiv 1$ collapses (5) to the ledger: the volume integrals vanish, and what is left is the plain sum $\sum_k \widehat{F}_k$ at each end. That is the whole mechanism behind the sum rule. Flags are private to their term; their *consequences* are shared, because every term is evaluated on the same f and adds into the same balance.

6 Mixing value and flux conditions across terms

Sections 4 and 5 can now be combined, and doing so exposes an asymmetry between the two kinds of condition.

- A **flux** condition (fixing a **div**) removes that term’s contribution from the ledger. Its effect is confined to its own term, and several of them combine additively — n sealed terms give the single condition $\sum_{k \in S} F_k = 0$, not n separate ones.
- A **value** condition (fixing a **grad**) does not remove a flux. It drives the shared unknown f to zero at that wall. Every other term then evaluates *its* flux on a solution that is being pinned, so a value condition imposed through one term is felt by all of them.

Two consequences follow, and they are the practical content of this section.

Advective terms may be left free at a Dirichlet wall. Their flux is $c f$, and $f \rightarrow 0$ there, so the flux extinguishes itself. This is why the recipe in §8 does not ask you to seal them: they are harmless, not overlooked. Note this holds for advection specifically, because its flux is proportional to the value.

Diffusive flux does *not* vanish at a Dirichlet wall. It is $D \partial_x f$, and $f \rightarrow 0$ says nothing about the slope — indeed the slope is generally steepest there. That surviving flux is precisely the absorption: it is the rate at which the wall consumes material. Sealing it would give a wall that holds $f = 0$ while admitting nothing, which is not absorption but a contradiction, and it is the ill-posed row below.

For an equation $\partial_t f = -\partial_x(Af + D \partial_x f)$ with the diffusion written as a chain, the combinations at one wall are:

adv. div	diff. div	diff. grad	condition at the wall	character
fixed	fixed	free	$Af + D \partial_x f = 0$	reflecting
free	free	fixed	$f = 0$	absorbing
fixed	free	fixed	$f = 0$	absorbing (sealing adv. is redundant)
free	fixed	free	$D \partial_x f = 0$	leaks Af — a <i>source</i> if A points in
fixed	free	free	$Af = 0$	leaks $D \partial_x f$
free	free	free	none	under-determined
free	fixed	fixed	$f = 0$ and $D \partial_x f = 0$	over-determined
fixed	fixed	fixed	as above	over-determined

The accounting rule that generates this table: a second-order problem admits one *independent* condition per wall. Sealed fluxes contribute one between them (their sum). A pinned value contributes one. Whether having both is fatal depends on whether they are independent:

- $f = 0$ together with $Af = 0$ is *redundant* — the first implies the second. Harmless; row 3.
- $f = 0$ together with $D \partial_x f = 0$ is *independent* — value and slope pinned at the same point. Over-determined; rows 7–8.

Stated generally, and not only for the two-term table: fixing a **grad** anywhere gives $f = 0$ at that wall. You may then additionally seal any *advective* terms — redundant, harmless — but sealing any *diffusive* flux, whether in that chain or in a different one, adds a genuinely independent condition and over-determines the wall. What matters is the kind of flux you seal, not whether it belongs to the same chain as the fixed **grad**.

Several grad conditions at once

These do *not* accumulate the way sealed **divs** do, and the contrast is the cleanest way to see what each kind of flag really is.

Sealing n fluxes gives one condition, $\sum_{k \in S} F_k = 0$, and *which* terms you put in S changes what that condition says: seal the drag alone and you get $f = 0$; seal drag and diffusion together and you get the Robin relation of §5. The F_k are different objects, so including more of them builds a different sum.

Fixing n **grads** gives one condition, $f = 0$, stated n times. Every **grad** flag reaches for a trace of the *same* field — the value $\hat{f}$, not a term-specific flux — and the only value it can be assigned is zero. A second fixed **grad** therefore cannot say anything the first did not, and cannot contradict it either:

$$\underbrace{\hat{f} = 0}_{\text{chain 1}} \quad \text{and} \quad \underbrace{\hat{f} = 0}_{\text{chain 2}} \quad \Longleftrightarrow \quad \hat{f} = 0. \quad (6)$$

Two **grad** flags never over-determine each other. What they do change is *stiffness*: each chain penalises $f(\text{wall}) \neq 0$ in proportion to its own diffusivity, so enforcement is as strong as $D_1 + D_2 + \dots$ rather than any one of them. Adding a second is a way to make an absorbing wall firmer, never a way to make it a different wall.

The asymmetry in one line: *flux flags compose, value flags coincide*.

A last remark on strength. The pinning is weak, not algebraic: it is imposed to the order of the scheme, not exactly, and if the responsible D is small the wall is only weakly absorbing. If the **grad** carries a coefficient g , what is pinned is $gf = 0$ — still $f = 0$ wherever $g \neq 0$, but nothing at all where g vanishes. Worth checking directly: coefficients like x^2 or $\sin \theta$ do vanish at a coordinate origin, and a flag placed there is decoration.

Penalty terms invert the convention

One exception, and it is a trap, because it inverts the rule everything above rests on. For a **div** or **grad**, **none** *writes* the boundary block (open) and a sealing flag *omits* it. For a **penalty** the assembler does the reverse:

term kind	none	bothsides/left/right
div, grad	block written from interior trace (open)	block omitted (flux = 0)
penalty	<i>no penalty applied</i> (inert)	penalty block added (Dirichlet)

The reason is that a penalty is not a flux. It penalises the jump $[f]$ across an interface, and at a domain edge there is no neighbour, hence no jump — so the natural thing at a free wall is to do nothing, which is what **none** selects. Giving a penalty a sealing flag instead makes it penalise $f - 0$, i.e. enforce $f = 0$: a Dirichlet penalty.

The practical consequence: *do not apply a uniform “seal everything” rule*. A penalty swept up in it stops being inert and becomes a value condition, which — placed alongside the sealed diffusive fluxes it was meant to accompany — lands you in the over-determined row of the table above. A zero-flux wall wants sealed **divs**, free **grads**, and penalties left at **none**.

7 Robin conditions

The two options in §2 are not the only ones. Instead of omitting the boundary block or filling it from the solution, a third choice writes a block of the prescribed form $r f(\text{wall})$. In the ledger this contributes $\mp r f$, a flux proportional to the local density: a partially absorbing wall, and the discrete form of $\hat{F} = r f$.

Two things follow immediately from the block structure. First, the coefficient r has the units of a flux per unit f — an effective velocity — not of a logarithmic derivative; supplying A/D

where r is wanted is off by a factor of the diffusivity. Second, since the free option writes $c f$ and this writes $r f$ into the same slot, choosing $r = -c$ cancels the free block exactly: *Robin with $r = -c(\text{wall})$ is identically the same operator as sealing that term.* If a code offers both, they are two spellings of one thing, and sealing is the safer spelling because it has no coefficient to get wrong.

Stability fixes the sign: the boundary term must remove energy, not add it, which requires r to describe outflow. Get it backwards and you have built positive feedback on the wall value.

8 Curvilinear coordinates and the Jacobian

If the conserved quantity is $\int m(x)f dx$ — $m = x^2$ in spherical speed, $x^2 \sin \theta$ in spherical velocity — then the equation being solved is $m \partial_t f = -\partial_x F$ and the flux is measured against that measure. The coefficient you hand a term is the *mass-weighted* flux coefficient, m times the physical one. This is why a physical drag $A = 1/(2x^2)$ appears in the code as the constant $1/2$: the x^2 of the Jacobian has cancelled it.

The practical consequence is that fluxes often vanish at a wall for geometric rather than physical reasons. At $x = 0$ the factor x^2 kills every flux, so the origin needs no condition. In a pitch-angle coordinate the $\sin \theta$ vanishes at $\theta = 0, \pi$ and the poles take care of themselves. The only walls that require thought are the ones where the measure does not vanish.

9 Choosing conditions for a new term

1. Write each term's flux, including the Jacobian factor.
2. At each wall, decide the direction: is that flux carrying material in or out?
3. Decide the physical law you want there: reflecting ($\sum F = 0$), absorbing ($f = 0$), partially absorbing ($F = r f$), or open outflow.
4. Reflecting: seal every term's flux — fix the **div** of each. For a chain, fix the **div** and leave the **grad** free.
5. Absorbing: fix the **grad** of the diffusive chain and leave its **div** free. Advective terms may stay free; $f \rightarrow 0$ at the wall makes their flux vanish with it.
6. Audit before running (§5): sum the unsealed fluxes and confirm it is the leak you meant.

Two diagnostics are worth building in. Track $\int m f$ over time: a steadily-drifting total means an unsealed flux you did not intend. And track $\max |f|$ as well, because an ill-posed pairing can conserve mass perfectly while diverging pointwise — conservation alone will not catch it.

10 Summary

- A boundary condition in DG is a choice of numerical flux at the two domain edges, nothing more. Interior edges cancel exactly, so the two end fluxes are the only route by which anything is gained or lost.

- In the matrix, the choice is which boundary block gets written: filled from the solution (free), omitted (zero flux), or prescribed (*rf*).
- A first-order term has one flux and one choice. A free edge on an *inflow* wall is a source, not a leak.
- A second-order term is a product of two factors, and the two integrations by parts give two traces: the outer factor owns the flux, the inner owns the value. Exactly one may be fixed per wall — flux gives Neumann, value gives Dirichlet, both is ill-posed, neither is under-determined.
- With several terms, only the sum of the unsealed fluxes leaks; sealing a subset imposes one condition on that subset's sum, which is how a Robin condition can appear without anyone writing one.